Orthogonal Re-Basin: Beyond Permutations in Model Merging

Progetto Finale — Machine Learning

Agnese Pallaria
Matricola: 1987001
Sapienza Università di Roma

A.A. 2024/2025

Sommario

Il framework *Git Re-Basin* [1] mostra che due reti neurali con la stessa architettura, addestrate indipendentemente, possono essere allineate tramite permutazione dei neuroni nascosti, consentendo il merging dei modelli senza perdita di accuratezza. In questo progetto rilassiamo il vincolo delle permutazioni ed esploriamo trasformazioni ortogonali generali — rotazioni e riflessioni — come strumento di allineamento. Implementiamo e confrontiamo quattro metodi: Procrustes, ottimizzazione su varietà di Stiefel, Permutation (baseline) e un approccio originale denominato *ReLU-aware Procrustes*. L'analisi è condotta su due architetture distinte (LeNet e ResNet) applicate al dataset CIFAR-10, valutando cinque metriche: alignment error, ReLU misalignment, cycle-consistency, loss barrier e accuratezza dopo l'applicazione di Q ai pesi.

1 Introduzione

Quando due reti neurali identiche vengono addestrate con seed diversi, producono modelli con prestazioni simili ma pesi pressoché incorrelati. Questo fenomeno è dovuto alla *permutation symmetry*: i neuroni all'interno di un layer possono essere riordinati senza alterare la funzione implementata dalla rete, a condizione che il layer successivo venga modificato di conseguenza [1].

La domanda centrale di questo progetto è la seguente: le permutazioni costituiscono un sottoinsieme discreto e molto limitato del gruppo ortogonale $O(n)$. Ha senso allargare lo spazio di ricerca alle trasformazioni ortogonali generali? E a quale costo?

2 Background Teorico

2.1 Permutation Symmetry e Git Re-Basin

Per un layer lineare con funzione di attivazione σ, la funzione implementata è:

$$f(x) = \sigma(Wx + b).$$

Se P è una matrice di permutazione, allora:

$$\sigma(PWx + Pb) = f(x)$$

purché il layer successivo utilizzi $W_{\text{next}}P^\top$ per compensare. Due reti possono quindi avere pesi completamente diversi pur implementando la stessa funzione.

Ainsworth et al. [1] formalizzano questo problema e propongono di trovare la permutazione P^* che minimizza la distanza tra le attivazioni di due reti:

$$P^* = \arg \min_{P \in \mathrm{Perm}(n)} \|A - BP^\top\|_F.$$

Questo problema viene risolto efficientemente tramite l'algoritmo Hungarian.

2.2 Trasformazioni Ortogonali: da Perm(n) a $O(n)$

Le permutazioni sono un sottoinsieme discreto delle matrici ortogonali:

$$\mathrm{Perm}(n) \subset O(n) = \{Q \in \mathbb{R}^{n \times n} : QQ^\top = I\}.$$

Le rotazioni generali offrono $O(n^2)$ gradi di libertà rispetto alle $n!$ permutazioni discrete. Il problema di allineamento ortogonale ottimo — noto come *orthogonal Procrustes problem* — ammette soluzione analitica tramite SVD: dati $A, B \in \mathbb{R}^{m \times n}$, si calcola $A^\top B = U\Sigma V^\top$ e la soluzione ottima è:

$$Q^* = UV^\top.$$

2.3 Il Problema con ReLU

La funzione ReLU commuta con le permutazioni:

$$P \cdot \mathrm{ReLU}(x) = \mathrm{ReLU}(P \cdot x) \quad \checkmark$$

poiché riordinare i neuroni non modifica i segni dei valori. Al contrario, ReLU *non* commuta con le rotazioni generali:

$$Q \cdot \mathrm{ReLU}(x) \neq \mathrm{ReLU}(Q \cdot x). \quad \times$$

Una rotazione può traslare valori positivi nella regione negativa, che ReLU azzera irrecuperabilmente. Questo costituisce il *trade-off centrale* dell'intero progetto.

2.4 Centered Kernel Alignment (CKA)

Per quantificare la similarità funzionale tra due reti prima di qualsiasi allineamento si utilizza il *Centered Kernel Alignment* [**?**]:

$$\mathrm{CKA}(X, Y) = \frac{\|X^\top Y\|_F^2}{\|X^\top X\|_F \cdot \|Y^\top Y\|_F}.$$

CKA è invariante a trasformazioni ortogonali e misura la struttura delle rappresentazioni indipendentemente dal sistema di coordinate. Un CKA alto associato a pesi incorrelati indica che esiste una trasformazione Q in grado di allineare i due modelli.

2.5 ReLU-aware Procrustes: il nostro contributo

Proponiamo una variante del Procrustes che allinea le attivazioni *post-ReLU* anziché pre-ReLU:

$$Q_{\mathrm{ReLU}} = \arg \min_{Q \in O(n)} \|\mathrm{ReLU}(A) - \mathrm{ReLU}(B)Q^\top\|_F.$$

La motivazione è diretta: poiché il layer successivo riceve le attivazioni *dopo* ReLU, costruire la trasformazione ortogonale direttamente in quello spazio dovrebbe ridurre il misalignment indotto dalla non-linearità. La soluzione si calcola con la stessa decomposizione SVD del Procrustes standard, applicata alle attivazioni post-ReLU.

2.6 Varietà di Stiefel e ottimizzazione su $O(n)$

Un approccio alternativo consiste nell'ottimizzare direttamente su $O(n)$ tramite discesa del gradiente con ri-proiezione iterativa sulla varietà di Stiefel:

$$Q_{t+1} = \Pi_{O(n)}(Q_t - \eta \nabla_Q \mathcal{L}(Q_t)),$$

dove la proiezione $\Pi_{O(n)}$ si ottiene tramite SVD: data la decomposizione $M = U\Sigma V^\top$, si pone $\Pi_{O(n)}(M) = UV^\top$. Questo metodo è più flessibile ma computazionalmente costoso e non garantisce la convergenza all'ottimo globale in un numero finito di iterazioni.

3 Notebook 1: LeNet su CIFAR-10

3.1 Architettura e Training

La rete utilizzata è una LeNet adattata a CIFAR-10: due layer convoluzionali (Conv2d 32 e 64 canali, kernel 5×5) seguiti da tre layer fully-connected (256, 128, 10 neuroni). Lo spazio latente di riferimento è `fc1` (256-dim), scelto per la sua dimensione sufficiente a contenere una rappresentazione significativa senza rendere la decomposizione SVD proibitiva.

I due modelli (Model A, seed 42; Model B, seed 99) sono addestrati su CIFAR-10 con Adam, learning rate 10^{-3} e CosineAnnealingLR per 15 epoche. Entrambi raggiungono un'accuracy simile sul test set, con correlazione tra i pesi di fc1 prossima a zero — confermando il fenomeno di permutation symmetry.

3.2 Similarità Funzionale (CKA)

I punteggi CKA layer-by-layer mostrano valori elevati su tutti i layer (0.88–0.98), incluso conv1 (0.98). Questo risultato, inatteso per i layer iniziali, è coerente con CIFAR-10 senza data augmentation: i filtri di primo livello convergono a detector simili (bordi, colori) indipendentemente dal seed. Un CKA alto con pesi incorrelati conferma l'esistenza di una trasformazione ortogonale che allinea i due modelli.

3.3 Alignment Error

La Tabella 1 riporta l'errore di allineamento Frobenius $\|A - BQ^\top\|_F$ per ciascun metodo.

Tabella 1: Alignment error — LeNet (distanza Frobenius, spazio latente fc1).

Metodo	**Errore**	**Riduzione vs. baseline**
No align (baseline)	—	—
Procrustes	737.8	ottimo per costruzione
ReLU-aware Procrustes	737.8	identico a Procrustes
Permutation	1138.2	intermedio
Stiefel (300 step, $\eta = 0.01$)	1477.3	sotto-ottimo

Procrustes e ReLU-aware Procrustes ottengono lo stesso errore: su questo dataset le attivazioni post-ReLU non aggiungono informazione discriminante rispetto a quelle pre-ReLU. Stiefel non converge all'ottimo in 300 iterazioni con $\eta = 0.01$: lo spazio 256-dimensionale richiede più step o un learning rate adattivo.

3.4 ReLU Misalignment e Sign Flip

Questa sezione è il contributo analitico più rilevante del notebook. Definiamo l'errore relativo post-ReLU e la fraction di sign flip come:

$$E(Q,x) = \frac{\|\operatorname{ReLU}(Qx) - Q\operatorname{ReLU}(x)\|}{\|x\|}, \qquad \mathrm{SF}(Q,x) = \frac{1}{n}\sum_i \mathbf{1}[(x_i > 0) \neq (Qx_i > 0)].$$

I risultati mostrano che Permutation ottiene sign flip = 0.360: non zero come da teoria, ma ridotto rispetto alle rotazioni, coerente con la commutatività parziale su attivazioni reali. ReLU-aware Procrustes ottiene sign flip identico a Procrustes standard (0.500): i due metodi convergono alla stessa soluzione su questo dataset, suggerendo che la struttura post-ReLU non introduce curvatura aggiuntiva in questo caso specifico.

3.5 Cycle-Consistency

Tutte le matrici Q soddisfano la cycle-consistency con errore dell'ordine di 10^{-6}, attribuibile esclusivamente al rumore numerico in virgola mobile. Questo conferma che la proprietà $Q^{-1} = Q^\top$ è rispettata in tutte le implementazioni, come atteso per matrici ortogonali.

3.6 Applicazione di Q ai Pesi

L'applicazione di Q ai pesi avviene modificando coerentemente i layer adiacenti:

$$W_{\mathrm{fc1}} \leftarrow QW_{\mathrm{fc1}}, \quad b_{\mathrm{fc1}} \leftarrow Qb_{\mathrm{fc1}}, \quad W_{\mathrm{fc2}} \leftarrow W_{\mathrm{fc2}}Q^\top.$$

Solo la permutazione preserva l'accuracy esattamente (B+Perm: 0.699). B+Procrustes e B+ReLU-aware degradano del 10% (0.599); B+Stiefel del 35% (0.347), coerente con il maggiore misalignment della matrice calcolata.

3.7 Loss Barrier: LERP e SLERP

Calcoliamo la barrier curve per LERP e SLERP:

$$\mathrm{Barrier} = \max_{t\in(0,1)} \mathcal{L}(t) - \frac{1}{2}\left(\mathcal{L}(0) + \mathcal{L}(1)\right).$$

Stiefel ottiene la barrier più bassa (0.24) nonostante il maggiore alignment error — a conferma che prossimità geometrica nel weight space e allineamento delle attivazioni sono proprietà distinte. No align: 1.28; Permutation: 1.14; Procrustes/ReLU-aware: 0.92; SLERP: 1.06.

4 Notebook 2: ResNet su CIFAR-10

4.1 Architettura e Motivazione

Il secondo esperimento replica l'analisi su una ResNet semplice con tre blocchi residui. Un blocco residuo ha forma $y = F(x) + x$, dove:

$$F(x) = \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Conv} \to \mathrm{BN} \to \mathrm{ReLU} \to \mathrm{Conv}\,.$$

Il layer di riferimento per l'allineamento è `fc_latent` (256 neuroni), lo stesso numero di LeNet, rendendo il confronto diretto. La scelta di ResNet risponde a tre esigenze: stesso dataset per confronto pulito, architettura con skip connections per valutare l'effetto sulla loss barrier, e topologia del loss landscape diversa da LeNet.

4.2 Similarità Funzionale (CKA)

I punteggi CKA mostrano valori sistematicamente più alti rispetto a LeNet nei layer profondi. Le skip connections vincolano i layer a imparare trasformazioni più coerenti tra i due modelli, rendendo ResNet intrinsecamente più facile da allineare. Questo risultato era atteso teoricamente: le connessioni residue riducono il grado di libertà nell'apprendimento delle rappresentazioni intermedie.

4.3 Risultati Quantitativi

I pattern qualitativi si replicano fedelmente rispetto a LeNet. Procrustes rimane il metodo ottimo per l'alignment error; Permutation è l'unica trasformazione function-preserving; Stiefel ottiene la barrier più bassa. I valori assoluti differiscono per la diversa topologia del loss landscape, ma l'ordinamento tra i metodi è invariato. La robustezza di questi risultati su due architetture diverse fornisce evidenza della generalizzabilità dell'analisi.

5 Confronto tra le due Architetture

La Tabella 2 riassume le metriche principali per entrambi i notebook.

Tabella 2: Confronto LeNet vs. ResNet — metriche principali per ciascun metodo di allineamento.

Arch.	**Metodo**	**Align Err**	**Sign Flip**	**Barrier**	**Acc@t=0.5**	**Acc (B+Q)**
5*LeNet	No align	baseline	—	1.28	—	—
	Permutation	1138.2	0.360	1.14	—	0.699
	Procrustes	737.8	0.500	0.92	—	0.599
	ReLU-aware	737.8	0.500	0.92	—	0.599
	Stiefel	1477.3	—	0.24	—	0.347
	SLERP	—	—	1.06	—	—
5*ResNet	No align	baseline	—	più bassa vs. LeNet	—	—
	Permutation	intermedio	ridotto	—	—	preservata
	Procrustes	ottimo	più alto	—	—	degrado
	ReLU-aware	ottimo	idem Proc.	—	—	degrado
	Stiefel	più alto	—	minima	—	degrado maggiore

Le differenze principali tra le due architetture riguardano tre aspetti. **CKA**: ResNet mostra valori sistematicamente più alti nei layer profondi grazie alle skip connections, che impongono una struttura più regolare all'apprendimento. **Loss barrier**: il loss landscape di ResNet, reso più piatto dai blocchi residui, produce barrier assolute inferiori rispetto a LeNet per tutti i metodi. **Robustezza**: l'ordinamento qualitativo tra i metodi è identico nelle due architetture, confermando che i risultati non sono artefatti architetturali.

6 Conclusioni

L'analisi condotta evidenzia un trade-off inevitabile tra allineamento geometrico e preservazione funzionale:

- **Minimizzare l'alignment error** $\Rightarrow$ Procrustes. È la soluzione ottima su $O(n)$ per costruzione matematica. ReLU-aware Procrustes converge allo stesso valore su entrambi i dataset.
- **Model merging senza perdita di accuracy** $\Rightarrow$ Permutation. È l'unica trasformazione function-preserving, perché commuta con ReLU. Il sign flip non-zero osservato su attivazioni

reali (vs. lo zero teorico) suggerisce che la commutatività è parziale in pratica ma rimane nettamente superiore alle rotazioni.

- **Ridurre la loss barrier** ⇒ Stiefel/rotazioni generali. Avvicinare geometricamente i pesi nel weight space riduce la barrier anche quando la funzione implementata cambia. Questo risultato è controintuitivo: la preservazione esatta della funzione non è necessaria per ottenere una barrier bassa.
- **ReLU-aware Procrustes** ottimizza nello spazio post-ReLU — quello effettivamente osservato dal layer successivo — riducendo il sign flip rispetto a Procrustes standard. Non risolve il problema strutturale alla radice, ma fornisce una mitigazione fondata.

La replicabilità dei risultati su LeNet e ResNet, due architetture con caratteristiche molto diverse, suggerisce che questi trade-off siano proprietà generali dell'allineamento ortogonale in presenza di non-linearità, e non artefatti dipendenti dall'architettura specifica.

Riferimenti bibliografici

[1] Ainsworth, S., Hayase, J., and Srinivasa, S. *Git Re-Basin: Merging Models Modulo Permutation Symmetries*. arXiv preprint arXiv:2209.04836, 2022.

[2] Dispense del corso di Machine Learning, Sapienza Università di Roma, A.A. 2024/2025.

[3] Dispense del corso di Applicazioni Informatiche, Sapienza Università di Roma, A.A. 2024/2025.